01 / The project at a glance

DocuChat is a PDF question-answering application. You upload a text PDF, select it, ask a question, and inspect the page excerpts referenced by the answer. This guide describes the local source as updated on 6 November 2026.

What is implemented

A single React workspace, light/dark preference, public shared document library, upload/select/delete actions, Gemini embeddings and answers, document-filtered search, and expandable citation references. Three fictional sample PDFs are bundled with the frontend and indexed only when a visitor chooses Use.

What this project demonstrates

Frontend state and HTTP integration, Python API design, PDF processing, vector retrieval, persistent storage choices, input validation, provider error handling and offline tests. Using a pretrained model through an API is inference integration; this project does not train or fine-tune a language model.

Evidence boundary

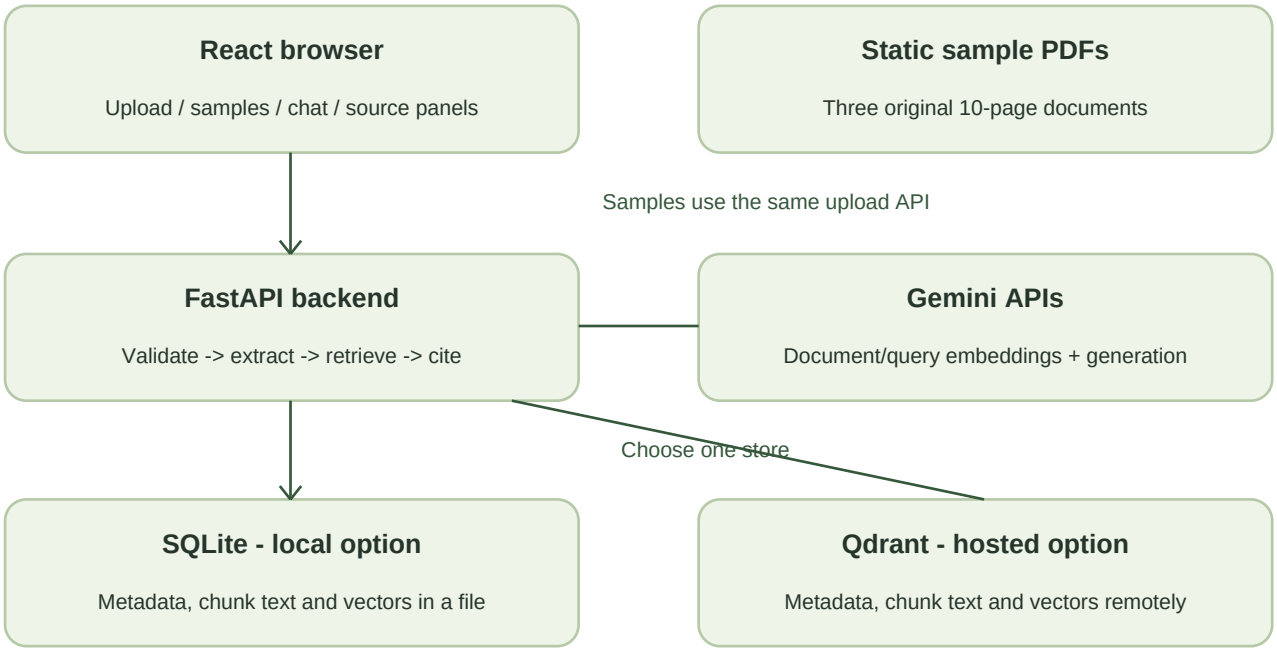
Before these notes, 42 offline backend tests passed and frontend lint/build passed. The sample additions receive separate checks. Offline tests mock Gemini and Qdrant HTTP calls. The current preview uses local SQLite. The updated UI and key removal have not been verified on the public Render deployment.

How to use these notes

Read the architecture first, then trace one upload and one question through the actual files. Run a check, change a small behavior and explain what changed. Treat planned improvements as proposals until they are implemented and measured.

02 / System architecture

RAG means retrieval-augmented generation: retrieve relevant document excerpts first, then give those excerpts to a language model when asking it to answer.



The browser calls FastAPI, not Gemini or Qdrant directly. Provider credentials belong on the backend. The sample PDFs are public static files; selecting one sends its bytes through the normal upload route.

The storage adapter selects SQLite or Qdrant. They are alternatives, not a replication pair. Qdrant stores extracted text, metadata and vectors; it does not store the original PDF. The current code does not retain chat history on the server.

Deployment mapping

Frontend	React/Vite static build on Render Static Site
Backend	FastAPI on Render Web Service, one worker
Hosted storage	Qdrant Cloud when configured
Model service	Gemini embedding and generation APIs

03 / Follow a PDF upload

Start in my-react-app/src/App.jsx at handleUpload or handleUseSample. Both send multipart form data to POST /upload. FormData is the browser object used to send a file without manually building MIME boundaries.

1. Validate before processing

backend/main.py bounds request bodies before multipart parsing and allows one modifying operation at a time with a process-local Lock. The upload handler checks the 5 MB file limit and the %PDF- signature. A signature is a quick filter, not proof that the file is valid.

2. Extract in another process

extract_pdf runs backend/pdf_extract.py as a subprocess and sends PDF bytes through stdin. pypdf extracts text page by page. The subprocess has a 25-second wall timeout; Linux adds memory/CPU limits. Encrypted, malformed and overly complex PDFs are rejected. Scanned image-only PDFs need OCR, which is not implemented.

3. Chunk and embed

chunk_text uses 120-word windows with 20-word overlap, plus a 1,600-byte cap for unusual text. Each chunk keeps its physical PDF page and a document-wide chunk number. embed_texts uses RETRIEVAL_DOCUMENT and 768-dimensional vectors. It batches up to twenty texts per request and normalizes valid vectors.

4. Publish the document

The handler creates a UUID document_id and records filename, pages, characters, chunks and embedding dimensions. The selected store writes chunk data and metadata. The UI reloads the library and selects the new document. No original PDF or server-side chat history is retained.

04 / Follow a question

handleSubmit sends JSON containing message and document_id to POST /chat. document_id is what keeps a question associated with one selected PDF.

Request validation

ChatRequest is a Pydantic model: message must have 1 to 2,000 characters and document_id must match 32 lowercase hexadecimal characters. The handler separately rejects whitespace-only questions. Schema validation protects the API even if a client bypasses the interface.

Retrieve once

retrieve_matches checks that the document exists, embeds the question with RETRIEVAL_QUERY, and calls store.search(document_id, vector, limit=3). Only the top three chunks from that document are returned. The code does not use hybrid search, reranking or an adaptive retry loop.

Generate from the retrieved text

backend/llm.py labels excerpts with chunk and page numbers. The prompt asks Gemini to use only those excerpts, ignore document instructions, and admit when information is absent. This is a behavioral instruction, not a guarantee that unsupported claims are impossible.

Validate references and render

cited_response checks cited chunk IDs and pages against retrieved matches. The frontend renders Markdown, cited sources and other retrieved context separately. Each chat request contains one question; previous messages are visible in the UI but are not sent as conversation memory.

```
POST /chat
{"message": "What is the CSV limit?",
 "document_id": "<32-character document identifier>"}
```

05 / Embeddings and retrieval

An embedding is a list of numbers representing a piece of text. A query vector is compared with stored chunk vectors to find passages that may answer the question. Retrieval selects evidence; generation writes the answer.

Why two task types?

Document chunks use RETRIEVAL_DOCUMENT; questions use RETRIEVAL_QUERY. Both use the same embedding model and 768 dimensions. A different model can produce a different vector space even with the same length, so changing embedding models requires reindexing documents.

Similarity and normalization

llm.py converts embedding values to floats, rejects non-finite values, checks length, rejects zero vectors, and divides by the vector norm. SQLite ranks by a dot product of normalized vectors. Qdrant uses a Cosine collection and a document_id/type filter.

Chunking tradeoff

A small chunk is cheap and specific but may omit necessary context. A large chunk includes more context but may dilute retrieval or cost more tokens. Overlap preserves nearby words across chunk boundaries. Page-by-page chunking keeps page citations understandable but can split ideas that span pages.

Important current limitation

Top-three retrieval can return irrelevant text when the answer is absent. There is no calibrated confidence cutoff. Similarity scores are not answer-accuracy percentages. Broad whole-document summaries can miss sections because the model receives only a few retrieved chunks.

06 / Citation checking

Read backend/citations.py carefully: its purpose is to validate reference identifiers, not to prove that a cited passage supports every sentence.

What the code checks

The available dictionary maps retrieved chunk numbers to their metadata. A regular expression finds bracketed references such as [Chunk 4, page 2]. A reference is retained only when its chunk was retrieved and its supplied page matches. A bare chunk reference can be resolved to that chunk's actual page.

What the API returns

reply contains the answer after invalid reference markers are removed. sources contains only the distinct retrieved excerpts actually cited, in citation order. other_context contains retrieved excerpts not cited. citation_warning explains invalid or missing references.

What checking does not prove

A real page can be cited beside an incorrect claim. Removing a fabricated citation marker does not remove or verify the associated claim. The code can therefore return answer text with a warning and no verified reference IDs. Do not describe this as factual validation or hallucination prevention.

A useful manual check

Use the employee sample and ask about remote days. The expected evidence is page 3. Check the answer, the page reference and the supporting excerpt. Then ask for the chief executive's salary, which is not provided. Record whether the model explicitly reports the information as absent.

07 / Storage and consistency

backend/storage.py supplies the same document operations for two storage choices. This lets the API call a stable interface while deployment configuration chooses the implementation.

SQLite for local work

SQLite stores documents and chunks in related tables inside one file. Chunk rows contain text, page numbers and serialized vectors. A transaction prevents partially inserted documents; foreign keys and cascade deletion keep document deletion consistent. Reopening the same retained file preserves the local library.

Qdrant for hosted indexing

Qdrant stores chunk points and a separate document manifest point. Search filters by document_id and type=chunk so the manifest never becomes a search result. Metadata listing uses paginated scroll queries without returning vector arrays.

Avoid half-published documents

Qdrant writes chunks first and publishes the manifest after successful writes. A failed insertion attempts cleanup; an absent manifest keeps orphan chunks out of the library/search flow. This is application-level publishing behavior, not a multi-request database transaction. Deletion removes chunks before the manifest so a failed chunk deletion remains retryable.

Retention is a separate question

Local files on Render Free are temporary. Hosted Qdrant separates the index from a backend restart, but account inactivity and provider retention still matter. The app has no backup/restore workflow or per-user library isolation. All visitors can read and delete shared documents.

08 / Frontend reading guide

my-react-app/src/App.jsx contains requestApi, the workspace state, upload/sample actions, document selection, chat submission and source rendering. App.css defines layout; index.css defines shared colors and light/dark themes.

State and references

useState stores values that change what React renders: documents, selectedDocumentId, messages, operation and theme. useRef stores mutable handles without causing a render: the active AbortController, file input and scroll target. A ref is useful for tracking an in-flight request between event callbacks.

Request lifecycle

runOperation disables conflicting actions, creates an AbortController and starts a timeout. It applies results only for the current component session, reports readable failures and clears busy state in finally. The initial library load runs in useEffect and aborts on unmount. These steps avoid stale results changing an abandoned view.

Theme and persistence

The sun/moon button changes theme state. A useEffect writes data-theme on the document root and stores docuchat-theme in localStorage. CSS variables supply both palettes. Only the theme preference is saved there; provider keys and chat history are not.

Sample behavior

The frontend bundles three ten-page PDFs and a small manifest listing title, filename and example question. Use reuses a same-name library entry when present; otherwise it fetches the static PDF and uploads it normally. The input placeholder changes to the selected sample's question. Typing a question still submits an ordinary /chat request.

09 / API and failure handling

The routes are public after removal of the shared demo key. Removing that key did not make the library private, add accounts or introduce user isolation.

GET /health	Process liveness only
GET /documents	Document metadata and storage notice
POST /upload	Multipart file field named file
POST /search	question + document_id; retrieved chunks only
POST /chat	message + document_id; answer and citations
DELETE /documents/{id}	Remove document and chunks

Bounds and error responses

The current code limits a PDF to 5 MB, 50 pages, 200,000 extracted characters and 250 chunks. The shared library is limited to twenty documents and 2,000 chunks. The middleware serializes POST/DELETE operations in one process. This is not per-IP rate limiting and does not coordinate across multiple workers.

Provider failures

Gemini quota errors map to 429, timeouts to 504, and selected provider/access errors to sanitized 502 messages. Storage failures return a controlled 503. Credentials are sent in request headers, not in provider URLs or frontend bundles. CORS controls browser origins; it is not authentication and does not prevent direct HTTP clients from calling a public API.

10 / Verification and deployment

Separate three kinds of evidence: offline checks, local live-provider checks and deployed acceptance checks. Passing one category does not establish the others.

Offline commands

Run backend/.venv/bin/python -m unittest discover -s backend/tests -v from the repository root. In my-react-app run npm run lint and npm run build. Backend tests exercise real PDF extraction and SQLite behavior, while Gemini/Qdrant HTTP contracts use mocks. The last verified backend run had 42 tests.

Live acceptance checklist

With valid provider configuration: upload two distinct PDFs, ask a question about each, inspect sources, switch documents, delete one and verify the other remains. Restart the backend and verify retained storage. Check a missing-answer question, a cold start and an oversized upload. Observe memory during indexing, not just at startup.

Render configuration

Frontend root: my-react-app; build: npm ci && npm run build; publish: dist; SPA rewrite to /index.html. Backend root: backend; install requirements; start uvicorn main:app --host 0.0.0.0 --port $PORT --workers 1. Set VITE_API_URL when building the frontend and FRONTEND_ORIGIN on the backend.

Backend-only configuration

GEMINI_API_KEY, GEMINI_MODEL, VECTOR_STORE, QDRANT_URL, QDRANT_API_KEY and QDRANT_COLLECTION configure the hosted services. Keep secrets outside source control. The generation-model default in this checkout is gemini-3.5-flash-lite; it is a code default, not proof that the account can access it. Updated public deployment remains pending.

11 / Interview explanations

Describe the system using one concrete request. Explain what you built, why a choice fits the constraints and which claims you have actually tested.

Why Gemini instead of self-hosting?

A hosted API keeps inference outside the lightweight backend and avoids maintaining model-serving hardware for the public demo. Free quota helped the budget. Tradeoffs include provider dependence, quotas, network latency and sending excerpts to an external service. No open-model benchmark has been completed, so do not claim Gemini is better.

Why no LangChain or LangGraph?

The current request flow is linear and implemented in separate Python functions. A framework is not needed for its present control flow. If a later feature needs saved workflow state, branching or human approval, evaluate orchestration then. Adding a framework alone would not establish retrieval quality.

What did you build around the model?

File validation and isolated extraction, chunk/page metadata, vector-store adapters, document filtering, citation-ID checking, failure handling, a React client and offline regression tests. The model and embedding service are pretrained external dependencies. Hosting an open-weight model would still be inference, not model training.

What would you improve next?

Build an answerable/unanswerable evaluation set and measure retrieval success, reference correctness, unsupported claims and latency. Compare one suitable open model under the same test questions. Add per-user isolation and usage controls only when the intended audience requires them. These are proposals, not implemented capabilities.

12 / Sample questions and practice

All three samples are original fictional documents with ten physical pages each. Their facts are deliberately stable so you can check retrieval and citations without uploading personal information.

Employee handbook	How many remote work days are allowed each week? Answer: Up to three remote work days per week, with Tuesday and Thursday in the office. Evidence: physical page 3.
Product specification	What is the maximum CSV import size in TaskBoard Lite? Answer: 2,000 task rows and 5 MB per CSV file. Evidence: physical page 6.
Project report	What was the p95 response time after the search optimization? Answer: 310 milliseconds, compared with 780 milliseconds before optimization. Evidence: physical page 7.

Questions with no answer

Handbook: What is the chief executive's salary? Product spec: What does the subscription cost? Project report: What was the user-satisfaction score? These facts are absent; the assistant should say so. The fictional project report's performance figures must never be presented as DocuChat results.

Exercises grounded in the code

1. Trace message from the input to ChatRequest and generate_answer. 2. Explain why search must filter by document_id. 3. Compare chunk sizes using the same ten questions. 4. Add a test for a concrete failure you reproduce. 5. Explain why a valid citation ID can accompany an unsupported claim. 6. Restart local storage and distinguish that check from hosted persistence.

Primary references

FastAPI documentation | React learning guide | Gemini embeddings | Qdrant documentation | Render Free limits. Consult these for current provider behavior; this guide describes the checked-out code, not an availability guarantee.